

ASPA – приложение ASP.NET CORE

♣ – задания, требующие самостоятельного изучения (нет в лекциях)

1. В лабораторной работе используется программный код, разработанный в предыдущих работах (**DAL004**, **ASPA004_x**)

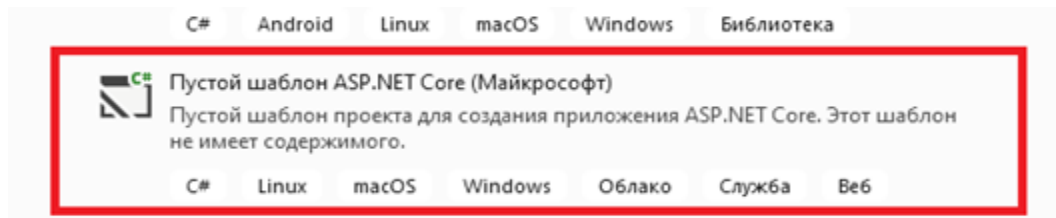
Задание 1.

2. Разработайте ASPA, применив следующий шаблон.

Имя решения: **ASPA**

Имя проекта: **ASPA005_1**

Версия .NET: **7.0** или **8.0**



3. Подключите библиотеку **DAL004** к **ASPA005_1**
4. Продублируйте программный код файла **Program.cs** приложения **ASPA004_3** в одноименный файл приложения **ASPA005_1**.
5. Разработайте три фильтра для конечной точки добавляющей данные о новой знаменитости. Фильтры предназначены для проверки входных данных. Назначение фильтров описывается в комментариях к следующему программному коду.

```
app.MapPost("/Celebrities", (Celebrity celebrity) => {
    int? id = repository.AddCelebrity(celebrity);
    if (id == null) throw new AddCelebrityException("POST /Celebrities error, id == null");
    if (repository.SaveChanges() <= 0) throw new SaveException("/Celebrities error, SaveChanges() <= 0");
    return new Celebrity((int)id, celebrity.Firstname, celebrity.Surname, celebrity.PhotoPath);
})
.AddEndpointFilter(async (context, next) => {
    // если параметр celebrity == null, то 500 с пояснением
    // если Surname == null или его длина < 2, то 409 с пояснением
    if (celebrity == null || celebrity.Surname == null || celebrity.Surname.Length < 2) {
        return await next(context);
    }
    return await next(context);
}).AddEndpointFilter(async (context, next) => {
    // если параметр celebrity == null, то 500 с пояснением
    // если уже есть такая Surname, то 409 с пояснением
    if (celebrity.Surname == null || celebrity.Surname.Length < 2) {
        return await next(context);
    }
    return await next(context);
}).AddEndpointFilter(async (context, next) => {
    // если параметр celebrity == null, то 500 с пояснением
    // если в BasePath нет файла с именем Path.GetFileName(celebrity.PhotoPath),
    // -- в ответе заголовок X-Celebrity: NotFound = Path.GetFileName(celebrity.PhotoPath)
    if (celebrity.PhotoPath == null || !Path.GetFileName(celebrity.PhotoPath).StartsWith(BasePath)) {
        return await next(context);
    }
    return await next(context);
})
```

6. Разработанный программный код фильтров, для обработки исключений должен использовать следующий механизм

```
//  
// app.UseExceptionHandler("/Celebrities/Error");  
// .....  
// app.Map("/Celebrities/Error", (HttpContext ctx) => {...});  
//
```

7. Разработанный программный код фильтров должен соответствовать следующим тестам.

POST http://localhost:5050/Celebrities

Params Authorization Headers (12) Body ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▾

```
1 {  
2   "firstname": "ForValidate",  
3   "photoPath": "/Photo/ForUpdate.jpg"  
4 }  
5  
6
```

Body Cookies Headers (7) Test Results 409 Conflict

{ } JSON ▾ Preview Visualize ▾

1 "Value:POST /Celebrities error, Surname is wrong"

POST http://localhost:5050/Celebrities

Params Authorization Headers (12) Body ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▾

```
1 {  
2   "firstname": "ForValidate",  
3   "surname": "",  
4   "photoPath": "/Photo/ForUpdate.jpg"  
5 }  
6
```

Body Cookies Headers (7) Test Results 409 Conflict

{ } JSON ▾ Preview Visualize ▾

1 "Value:POST /Celebrities error, Surname is wrong"

POST http://localhost:5050/Celebrities

Params Authorization Headers (12) Body ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▾

```
1 {  
2   "firstname": "ForValidate",  
3   "surname": "x",  
4   "photoPath": "/Photo/ForUpdate.jpg"  
5 }  
6
```

Body Cookies Headers (7) Test Results 409 Conflict

{ } JSON ▾ Preview Visualize ▾

1 "Value:POST /Celebrities error, Surname is wrong"

HTTP Celebrities / <http://localhost:5050/Celebrities>

POST <http://localhost:5050/Celebrities>

Params Authorization Headers (12) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   |
3   |   "firstname": "ForValidate",
4   |   "surname": "xx",
5   |   "photoPath": "/Photo/ForUpdate.jpg"
6 }
```

Body Cookies Headers (4) Test Results **200 OK**

{ } JSON ▾ ▶ Preview Visualize ▾

```
1 {
2   |
3   |   "id": 9,
4   |   "firstname": "ForValidate",
5   |   "surname": "xx",
6   |   "photoPath": "/Photo/ForUpdate.jpg"
7 }
```

POST <http://localhost:5050/Celebrities/>

Params Authorization Headers (12) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   |
3   |   "firstname": "Noam",
4   |   "surname": "Chomsky",
5   |   "photoPath": "/Photo/Chomsky.jpg"
6 }
```

Body Cookies Headers (7) Test Results **409 Conflict**

{ } JSON ▾ ▶ Preview Visualize ▾

```
1 "Value:POST /Celebrities error, Surname is doubled"
```

HTTP Celebrities / <http://localhost:5050/Celebrities/9>

DELETE <http://localhost:5050/Celebrities/1>

Params Authorization Headers (7) **Body** Scripts Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers (4) Test Results **200 OK**

{ } JSON ▾ ▶ Preview Visualize ▾

```
1 "Celebrity with Id = 1 deleted"
```

POST http://localhost:5050/Celebrities/

Params Authorization Headers (12) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   |
3   |   "firstname": "Noam",
4   |   "surname": "Chomsky",
5   |   "photoPath": "/Photo/Chomsky.jpg"
6 }
7
8
9
```

Body Cookies Headers (4) Test Results 200 OK

{ } JSON Preview Visualize

```
1 {
2   |
3   |   "id": 13,
4   |   "firstname": "Noam",
5   |   "surname": "Chomsky",
6   |   "photoPath": "/Photo/Chomsky.jpg"
7 }
```

POST http://localhost:5050/Celebrities/

Params Authorization Headers (12) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   |
3   |   "firstname": "Noam",
4   |   "surname": "Chomsky",
5   |   "photoPath": "/Photo/Chomsky.jpg"
6 }
7
8
9
```

Body Cookies Headers (4) Test Results 200 OK

Key	Value
Content-Type	application/json; charset=utf-8
Date	Mon, 24 Feb 2025 10:39:56 GMT
Server	Kestrel
Transfer-Encoding	chunked

HTTP Celebrities / ForDelete http://localhost:5050/Celebrities

POST http://localhost:5050/Celebrities/

Params Authorization Headers (12) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   |
3   |   "firstname": "NoJPG",
4   |   "surname": "NoJPG",
5   |   "photoPath": "/Photo/NoJPG.jpg"
6 }
7
8
9
```

Body Cookies Headers (5) Test Results 200 OK

{ } JSON Preview Visualize

```
1 {
2   |
3   |   "id": 14,
4   |   "firstname": "NoJPG",
5   |   "surname": "NoJPG",
6   |   "photoPath": "/Photo/NoJPG.jpg"
7 }
```

POST http://localhost:5050/Celebrities/

Params Authorization Headers (12) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```

1 {
2   |
3   |   "firstname": "NoJPG",
4   |   "surname": "NoJPG",
5   |   "photoPath": "/Photo/NoJPG.jpg"
6   | }
7
8
9

```

Body Cookies Headers (5) Test Results 200 OK

Key	Value
Content-Type	application/json; charset=utf-8
Date	Mon, 24 Feb 2025 10:48:12 GMT
Server	Kestrel
Transfer-Encoding	chunked
X-Celebrity	NotFound=NoJPG.jpg

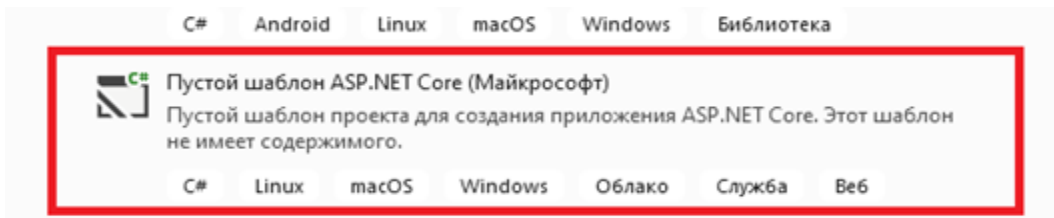
Задание 2.

8. Разработайте ASPA, применив следующий шаблон.

Имя решения: **ASPA**

Имя проекта: **ASPA005_2**

Версия .NET: **7.0** или **8.0**



9. Подключите библиотеку **DAL004** к **ASPA005_2**

10. Продублируйте программный код файла **Program.cs** приложения **ASPA005_1** в одноименный файл приложения **ASPA005_2**.

11. Перепишите код фильтров, разработанных в **ASPA005_1** в следующем новом виде, с применением **AddEndpointFilter<>** и классов фильтров.

```

Validation.SurnameFilter.repository = Validation.PhotoExistFilter.repository = repository;

app.MapPost("/Celebrities", (Celebrity celebrity) => {
    // ... (code from ASPA005_1) ...
    // 1. (repository, savechanges()) -> 0;
    // 2. (repository, savechanges()) -> 0;
    // 3. (repository, savechanges()) -> 0;
    return new Celebrity((int)0, celebrity.firstname, celebrity.surname, celebrity.photoPath);
})
.AddEndpointFilter<Validation.SurnameFilter>() // объединены 2 проверки: на пустое и дублирующееся имя
.AddEndpointFilter<Validation.PhotoExistFilter>(); // проверка наличия файла фотографии

```

12. Следует обратить внимание на следующее:

- 1) для передачи в классы фильтров ссылки на объект репозитория следует применить static-поле в этих классах, для передачи ссылки на репозиторий;
- 2) обработка исключений осуществляется по схеме

```
//  
// app.UseExceptionHandler("/Celebrities/Error");  
// .....  
// app.Map("/Celebrities/Error", (HttpContext ctx) => {....});  
//
```

- 3) в приведённом примере, фильтр, реализованный классом **SurnameFilter**, объединяет в себе функциональность двух проверок, выполняемых **ASPA005_1**.

13. Обеспечьте работу **ASPA005_2** соответствующую тестам для **ASPA005_1**.

14. ♣ Разработайте по одному классу фильтра для конечных точек **MapPut** и **MapDelete**.

15. Примените следующую конструкцию для создания общей группы маршрутов в **ASPA005_2**. Проверьте работоспособность маршрутов.

```
RouteGroupBuilder api = app.MapGroup("/Celebrities");
```

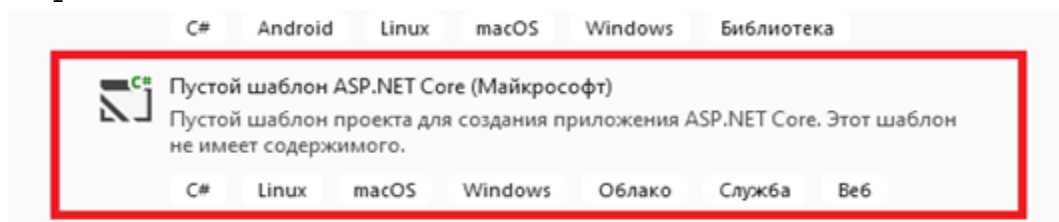
Задание 3.

16. Разработайте ASPA, применив следующий шаблон.

Имя решения: **ASPA**

Имя проекта: **ASPA005_3**

Версия .NET: **7.0** или **8.0**



17. Исследуйте таблицу, описывающую констрейнты для маршрутов, применяемых в ASPA.

constraint	Example	Example Matches	Notes
int	{id:int}	123456789, -123456789	Matches any integer
bool	{active:bool}	true, FALSE	Matches true or false. Case-insensitive
datetime	{dob:datetime}	2016-12-31, 2016-12-31 7:32pm	Matches a valid DateTime value in the invariant culture. See preceding warning.
decimal	{price:decimal}	49.99, -1,000.01	Matches a valid decimal value in the invariant culture. See preceding warning.
double	{weight:double}	1.234, -1,001.01e8	Matches a valid double value in the invariant culture. See preceding warning.
float	{weight:float}	1.234, -1,001.01e8	Matches a valid float value in the invariant culture. See preceding warning.
guid	{id:guid}	CD2C1638-1638-72D5-1638-DEADBEEF1638	Matches a valid Guid value
long	{ticks:long}	123456789, -123456789	Matches a valid long value
minlength(value)	{username:minlength(4)}	Rick	String must be at least 4 characters
maxlength(value)	{filename:maxlength(8)}	MyFile	String must be no more than 8 characters
length(length)	{filename:length(12)}	somefile.txt	String must be exactly 12 characters long
length(min,max)	{filename:length(8,16)}	somefile.txt	String must be at least 8 and no more than 16 characters long
min(value)	{age:min(18)}	19	Integer value must be at least 18
max(value)	{age:max(120)}	91	Integer value must be no more than 120
range(min,max)	{age:range(18,120)}	91	Integer value must be at least 18 but no more than 120
alpha	{name:alpha}	Rick	String must consist of one or more alphabetical characters, a-z and case-insensitive.
regex(expression)	{ssn:regex(^\\d{3}-\\d{2}-\\d{4}\$)}	123-45-6789	String must match the regular expression. See tips about defining a regular expression.
required	{name:required}	Rick	Used to enforce that a non-parameter value is present during URL generation

18. Применяя констрейнты создайте в **ASPA005_3** конечные точки, описанные в следующей таблице.

метод	маршрут	ограничения
A		
GET	/A/x	x = целочисленное значение <=100, y = null

POST	/A/x	x = целочисленное значение $0 \leq x \leq 100$, y = null
PUT	/A/x/y	x = целочисленное значение ≥ 1 , y = целочисленное значение ≥ 1
DELETE	/A/x-y	x = целочисленное значение ≥ 1 , y = целочисленное значение ≥ 1 , ≤ 100
В		
GET	/B/x	x = float-значение, y = null
POST	/B/x/y	x = float-значение, y = float-значение
DELETE	/B/x-y	x = float-значение, y = float-значение
С		
GET	/C/x	x = bool-значение, y = null
POST	/C/x,y	x = bool-значение, y = bool-значение
Д		
GET	/D/x	x = DateTime-значение, y = null
POST	/D/x y	x = DateTime-значение, y = DateTime-значение
Е		
GET	/E/12-x	x = строка, обязательное значение y=null
PUT	/E/x	x = строка, только буквы, $2 \leq \text{длина} \leq 12$ y=null
Ф		
PUT	/F/x	x = строка, формат email, домен=by y=null

19. Ниже приведён фрагмент серверного программного кода, реализующий конечные точки с заданными маршрутами.

```
app.UseExceptionHandler("/Error");

//--- А -----
app.MapGet("/A/{x:int}", (HttpContext context, [FromRoute] int? x) => Results.Ok(new { path = context.Request.Path.Value, x = x }));
app.MapPost("/A/{x:int}/{y:int}", (HttpContext context, [FromRoute] int x) => Results.Ok(new { path = context.Request.Path.Value, x = x }));
app.MapPut("/A/{x:int}/{y:int}", (HttpContext context, [FromRoute] int x, [FromRoute] int y) => Results.Ok(new { path = context.Request.Path.Value, x = x, y = y }));
app.MapDelete("/A/{x:int}/{y:int}", (HttpContext context, [FromRoute] int x, [FromRoute] int y) => Results.Ok(new { path = context.Request.Path.Value, x = x, y = y }));
//--- В -----
app.MapGet("/B/{x:float}", (HttpContext context, [FromRoute] float x) => Results.Ok(new { path = context.Request.Path.Value, x = x }));
app.MapPost("/B/{x:float}/{y:float}", (HttpContext context, [FromRoute] float x, [FromRoute] float y) => Results.Ok(new { path = context.Request.Path.Value, x = x, y = y }));
app.MapDelete("/B/{x:float}/{y:float}", (HttpContext context, [FromRoute] float x, [FromRoute] float y) => Results.Ok(new { path = context.Request.Path.Value, x = x, y = y }));
//--- С -----
app.MapGet("/C/{x:bool}", (HttpContext context, [FromRoute] bool x) => Results.Ok(new { path = context.Request.Path.Value, x = x }));
app.MapPost("/C/{x:bool}/{y:bool}", (HttpContext context, [FromRoute] bool x, [FromRoute] bool y) => Results.Ok(new { path = context.Request.Path.Value, x = x, y = y }));
//--- Д -----
app.MapGet("/D/{x:datetime}", (HttpContext context, [FromRoute] DateTime x) => Results.Ok(new { path = context.Request.Path.Value, x = x }));
app.MapPost("/D/{x:datetime}/{y:datetime}", (HttpContext context, [FromRoute] DateTime x, [FromRoute] DateTime y) => Results.Ok(new { path = context.Request.Path.Value, x = x, y = y }));
//--- Е -----
app.MapGet("/E/{x:string}", (HttpContext context, [FromRoute] string x) => Results.Ok(new { path = context.Request.Path.Value, x = x }));
app.MapPut("/E/{x:string}", (HttpContext context, [FromRoute] string x) => Results.Ok(new { path = context.Request.Path.Value, x = x }));
//--- Ф -----
app.MapGet("/F/{x:string}", (HttpContext context, [FromRoute] string x) => Results.Ok(new { path = context.Request.Path.Value, x = x }));

app.MapFallback((HttpContext ctx) => {return Results.NotFound(new { message = $"path {ctx.Request.Path.Value} not supported" });});

app.Map("/Error", (HttpContext ctx) => {
    Exception? ex = ctx.Features.Get<ExceptionHandlerFeature>().Error;
    return Results.Ok(new { message = ex?.Message });
});
```

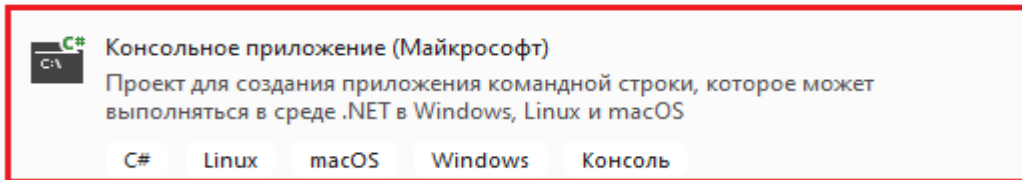
Задание 4.

20. Разработайте консольное приложение, применив следующий шаблон

Имя решения: **ASPA**

Имя проекта: **Test_ASPA005_3**

Версия .NET: **7.0** или **8.0**



21. Приложение **Test_ASPA005_3** является http-клиентом и предназначено для тестирования **ASPA005_3**.

22. Ниже в качестве примера приведены фрагменты программного кода и результат теста.

```
Test test = new Test();

Console.WriteLine("-- /A -----");
await test.ExecuteGET<int?>("http://localhost:5000/A/3", (int? x, int? y, int status) => (x == 3 && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteGET<int?>("http://localhost:5000/A/-3", (int? x, int? y, int status) => (x == -3 && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteGET<int?>("http://localhost:5000/A/118", (int? x, int? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecutePOST<int?>("http://localhost:5000/A/5", (int? x, int? y, int status) => (x == 5 && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecutePOST<int?>("http://localhost:5000/A/-5", (int? x, int? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecutePOST<int?>("http://localhost:5000/A/118", (int? x, int? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecutePUT<int?>("http://localhost:5000/A/2/3", (int? x, int? y, int status) => (x == 2 && y == 3 && status == 200) ? Test.OK : Test.NOK);
await test.ExecutePUT<int?>("http://localhost:5000/A/0/3", (int? x, int? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecutePUT<int?>("http://localhost:5000/A/25/-3", (int? x, int? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecutePUT<int?>("http://localhost:5000/A/0/-3", (int? x, int? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecuteDELETE<int?>("http://localhost:5000/A/1-99", (int? x, int? y, int status) => (x == 1 && y == 99 && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteDELETE<int?>("http://localhost:5000/A/99-1", (int? x, int? y, int status) => (x == 99 && y == 1 && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteDELETE<int?>("http://localhost:5000/A/-1-25", (int? x, int? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecuteDELETE<int?>("http://localhost:5000/A/-1-25", (int? x, int? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecuteDELETE<int?>("http://localhost:5000/A/25-101", (int? x, int? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);

Console.WriteLine("-- /B -----");
await test.ExecuteGET<float?>("http://localhost:5000/B/2.5", (float? x, float? y, int status) => (x == 2.5 && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteGET<float?>("http://localhost:5000/B/2", (float? x, float? y, int status) => (x == 2.0 && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteGET<float?>("http://localhost:5000/B/2X", (float? x, float? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecutePOST<float?>("http://localhost:5000/B/2.5/3.2", (float? x, float? y, int status) => (x == 2.5 && y == 3.2 && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteDELETE<float?>("http://localhost:5000/B/2.5-3.2", (float? x, float? y, int status) => (x == 2.5 && y == 3.2 && status == 200) ? Test.OK : Test.NOK);

Console.WriteLine("-- /C -----");
await test.ExecuteGET<bool?>("http://localhost:5000/C/2.5", (bool? x, bool? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecuteGET<bool?>("http://localhost:5000/C/true", (bool? x, bool? y, int status) => (x == true && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecutePOST<bool?>("http://localhost:5000/C/true,false", (bool? x, bool? y, int status) => (x == true && y == false && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteDELETE<bool?>("http://localhost:5000/C/true,false", (bool? x, bool? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);

Console.WriteLine("-- /D -----");
await test.ExecuteGET<DateTime?>("http://localhost:5000/D/2025-02-25", (DateTime? x, DateTime? y, int status) => (x == new DateTime(2025, 02, 25) && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteGET<DateTime?>("http://localhost:5000/D/2025-02-29", (DateTime? x, DateTime? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecuteGET<DateTime?>("http://localhost:5000/D/2024-02-29", (DateTime? x, DateTime? y, int status) => (x == new DateTime(2024, 02, 29) && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteGET<DateTime?>("http://localhost:5000/D/2025-02-25T19:25", (DateTime? x, DateTime? y, int status) => (x == new DateTime(2025, 02, 25, 19, 25, 0) && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecutePOST<DateTime?>("http://localhost:5000/D/2025-02-25T19:25", (DateTime? x, DateTime? y, int status) => (x == new DateTime(2025, 02, 25) && y == new DateTime(2025, 03, 25) && status == 200) ? Test.OK : Test.NOK);
await test.ExecutePUT<DateTime?>("http://localhost:5000/D/2025-02-25T19:25", (DateTime? x, DateTime? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);

Console.WriteLine("-- /E -----");
await test.ExecuteGET<string?>("http://localhost:5000/E/12-bis", (string? x, string? y, int status) => (x == "bis" && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteGET<string?>("http://localhost:5000/E/11-bis", (string? x, string? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecuteGET<string?>("http://localhost:5000/E/12-777", (string? x, string? y, int status) => (x == "777" && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteGET<string?>("http://localhost:5000/E/12-", (string? x, string? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecutePUT<string?>("http://localhost:5000/E/abcd", (string? x, string? y, int status) => (x == "abcd" && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecutePUT<string?>("http://localhost:5000/E/abcd123", (string? x, string? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecutePUT<string?>("http://localhost:5000/E/a", (string? x, string? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecutePUT<string?>("http://localhost:5000/E/123456", (string? x, string? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecutePUT<string?>("http://localhost:5000/E/aabbccddeeffgghh", (string? x, string? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);

Console.WriteLine("-- /F -----");
await test.ExecuteGET<string?>("http://localhost:5000/F/smw@belstu.by", (string? x, string? y, int status) => (x == "smw@belstu.by" && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteGET<string?>("http://localhost:5000/F/xxx@yyyy.by", (string? x, string? y, int status) => (x == "xxx@yyyy.by" && y == null && status == 200) ? Test.OK : Test.NOK);
await test.ExecuteGET<string?>("http://localhost:5000/F/xxx@yyyy.ru", (string? x, string? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecuteGET<string?>("http://localhost:5000/F/xxx@yyyy.by", (string? x, string? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
await test.ExecuteGET<string?>("http://localhost:5000/F/xxx@yyyy", (string? x, string? y, int status) => (x == null && y == null && status == 404) ? Test.OK : Test.NOK);
```

```

class Test
{
    class Answer<T>
    {
        public T? x { get; set; } = default(T?);
        public T? y { get; set; } = default(T?);
        public string? message { get; set; } = null;
    }
    public static string OK = "OK ", NOK = "NOK";
    HttpClient client = new HttpClient();
    public async Task ExecuteGET<T>(string path, Func<T?, T?, int, string> result)
    {
        await resultPRINT<T>("GET", path, await this.client.GetAsync(path), result);
    }
    public async Task ExecutePOST<T>(string path, Func<T?, T?, int, string> result)
    {
        await resultPRINT<T>("POST", path, await this.client.PostAsync(path, null), result);
    }
    public async Task ExecutePUT<T>(string path, Func<T?, T?, int, string> result)
    {
        await resultPRINT<T>("PUT", path, await this.client.PutAsync(path, null), result);
    }
    public async Task ExecuteDELETE<T>(string path, Func<T?, T?, int, string> result)
    {
        await resultPRINT<T>("DELETE", path, await this.client.DeleteAsync(path), result);
    }
    async Task resultPRINT<T>(string method, string path, HttpResponseMessage rm, Func<T?, T?, int, string> result)
    {
        int status = (int)rm.StatusCode;
        try
        {
            Answer<T>? answer = await rm.Content.ReadFromJsonAsync<Answer<T>>()??default(Answer<T>);
            string r = result(default(T), default(T), status);
            T? x = default(T), y = default(T);
            if (answer != null) r = result(x = answer.x, y = answer.y, status);
            Console.WriteLine($" {r}: {method} {path}, status = {status}, x = {x}, y = {y}, m = {answer?.message}");
        }
        catch (JsonException ex)
        {
            string r = result(default(T), default(T), status);
            Console.WriteLine($" {r}: {method} {path}, status = {status}, x = {null}, y = {null}, m = {ex.Message}");
        }
    }
}

```

```

-- /A -----
OK : GET http://localhost:5000/A/3, status = 200, x = 3, y = , m =
OK : GET http://localhost:5000/A/-3, status = 200, x = -3, y = , m =
OK : GET http://localhost:5000/A/118, status = 404, x = , y = , m = path /A/118 not supported
OK : POST http://localhost:5000/A/5, status = 200, x = 5, y = , m =
OK : POST http://localhost:5000/A/-5, status = 404, x = , y = , m = path /A/-5 not supported
OK : POST http://localhost:5000/A/118, status = 404, x = , y = , m = path /A/118 not supported
OK : PUT http://localhost:5000/A/2/3, status = 200, x = 2, y = 3, m =
OK : PUT http://localhost:5000/A/0/3, status = 404, x = , y = , m = path /A/0/3 not supported
OK : PUT http://localhost:5000/A/25/-3, status = 404, x = , y = , m = path /A/25/-3 not supported
OK : PUT http://localhost:5000/A/0/-3, status = 404, x = , y = , m = path /A/0/-3 not supported
OK : DELETE http://localhost:5000/A/1-99, status = 200, x = 1, y = 99, m =
OK : DELETE http://localhost:5000/A/99-1, status = 200, x = 99, y = 1, m =
OK : DELETE http://localhost:5000/A/-1-25, status = 404, x = , y = , m = path /A/-1-25 not supported
OK : DELETE http://localhost:5000/A/-1--25, status = 404, x = , y = , m = path /A/-1--25 not supported
OK : DELETE http://localhost:5000/A/25-101, status = 404, x = , y = , m = path /A/25-101 not supported
-- /B -----
OK : GET http://localhost:5000/B/2.5, status = 200, x = 2,5, y = , m =
OK : GET http://localhost:5000/B/2, status = 200, x = 2, y = , m =
OK : GET http://localhost:5000/B/2X, status = 404, x = , y = , m = path /B/2X not supported
NOK: POST http://localhost:5000/B/2.5/3.2, status = 200, x = 2,5, y = 3,2, m =
NOK: DELETE http://localhost:5000/B/2.5-3.2, status = 200, x = 2,5, y = 3,2, m =
-- /C -----
OK : GET http://localhost:5000/C/2.5, status = 404, x = , y = , m = The input does not contain any JSON tokens. Expected the input
to start with a valid JSON token, when isFinalBlock is true. Path: $ | LineNumber: 0 | BytePositionInLine: 0.
OK : GET http://localhost:5000/C/true, status = 200, x = True, y = , m =
OK : POST http://localhost:5000/C/true,false, status = 200, x = True, y = False, m =
OK : DELETE http://localhost:5000/C/true,false, status = 404, x = , y = , m = path /C/true,false not supported
-- /D -----
OK : GET http://localhost:5000/D/2025-02-25, status = 200, x = 25.02.2025 0:00:00, y = , m =
OK : GET http://localhost:5000/D/2025-02-29, status = 404, x = , y = , m = path /D/2025-02-29 not supported
OK : GET http://localhost:5000/D/2024-02-29, status = 200, x = 29.02.2024 0:00:00, y = , m =
OK : GET http://localhost:5000/D/2025-02-25T19:25, status = 200, x = 25.02.2025 19:25:00, y = , m =
OK : POST http://localhost:5000/D/2025-02-25|2025-03-25, status = 200, x = 25.02.2025 0:00:00, y = 25.03.2025 0:00:00, m =
OK : PUT http://localhost:5000/D/2025-02-25T19:25, status = 404, x = , y = , m = path /D/2025-02-25T19:25 not supported

```

```

-- /E -----
OK : GET http://localhost:5000/E/12-bis, status = 200, x = bis, y = , m =
OK : GET http://localhost:5000/E/11-bis, status = 404, x = , y = , m = path /E/11-bis not supported
OK : GET http://localhost:5000/E/12-777, status = 200, x = 777, y = , m =
OK : GET http://localhost:5000/E/12-, status = 404, x = , y = , m = path /E/12- not supported
OK : PUT http://localhost:5000/E/abcd, status = 200, x = abcd, y = , m =
OK : PUT http://localhost:5000/E/abcd123, status = 404, x = , y = , m = path /E/abcd123 not supported
OK : PUT http://localhost:5000/E/a, status = 404, x = , y = , m = path /E/a not supported
OK : PUT http://localhost:5000/E/123456, status = 404, x = , y = , m = path /E/123456 not supported
OK : PUT http://localhost:5000/E/aabbccddeeffgghh, status = 404, x = , y = , m = path /E/aabbccddeeffgghh not supported
-- /F -----
OK : GET http://localhost:5000/F/smw@belstu.by, status = 200, x = smw@belstu.by, y = , m =
OK : GET http://localhost:5000/F/xxx@yyyy.by, status = 200, x = xxx@yyyy.by, y = , m =
OK : GET http://localhost:5000/F/xxx@yyyy.ru, status = 404, x = , y = , m = The input does not contain any JSON tokens. Expected t
e input to start with a valid JSON token, when isFinalBlock is true. Path: $ | LineNumber: 0 | BytePositionInLine: 0.
OK : GET http://localhost:5000/F/xxxxxyyy.by, status = 404, x = , y = , m = The input does not contain any JSON tokens. Expected t
input to start with a valid JSON token, when isFinalBlock is true. Path: $ | LineNumber: 0 | BytePositionInLine: 0.
OK : GET http://localhost:5000/F/xxx@yyyy, status = 404, x = , y = , m = path /F/xxx@yyyy not supported

```

23. Реализуйте приложение **Test_ASPA005_3** для тестирования серверных конечных точек **ASPA005_3**.

24. Выполните тесты аналогичные представленные выше и убедитесь в совпадении результатов.

Задание 5. Ответьте на вопросы

25. Поясните понятие: фильтры конечной точки, принципы их действия.

26. Поясните понятие: констрейнт маршрута.

27. Перечислите известные вам констрейнты маршрутов и принцип их действия.